

Clash of Codes

Real-Time Competitive Coding & Strategy Game

Updated GDD, Migration Guide & 4-Week Build Plan

Version 3.0 — Picking up from Day 4

Project 1 — API

ASP.NET Core 8 Web API
EF Core + SQLite
JWT Auth + SignalR Hub
Judge0 + Matchmaking

Project 2 — Client

Blazor Server (existing)
HttpClient + SignalR client
Monaco Editor (JS interop)
Azure App Service

Connected via **REST (HTTP/J-
SON)** and **WebSockets (SignalR)**

Current status: **Day 4 complete** — Mod-
els, DbContext, Migrations, Admin page built

Next step: **Migration to two-project
architecture**, then resume Day 5

2026

This document covers the current project state, the migration steps required to split into two projects, and the complete revised day-by-day build plan resuming from Day 5.

Contents

Abstract	3
1 The Core Game Loop & Resource Economy	4
1.1 Battle Elixir (BE)	4
1.2 Rank Points (RP) and the Conversion Mechanic	4
2 Match Structure & Victory Conditions	4
2.1 The Three-Track Node Setup	4
2.2 Node Workflow	4
3 The Spellbook Mechanics	5
3.1 Offensive Spells	5
3.2 Defensive Spells	5
4 Matchmaking & Arenas	6
4.1 Ranked Ladder	6
4.2 Custom Lobbies	6
4.3 Syntax Stages (Arenas)	6
5 Recommended Design Improvements	7
5.1 Spell Cooldown	7
5.2 Active Debuff Cap	7
5.3 Disconnect Handling	7
5.4 Spell Notifications	7
6 Current Project State (End of Day 4)	8
6.1 What You Have and Where It Goes	8
7 Migration Guide (Day 4.5)	8
Step 1: Create the API project	8
Step 2: Move models, DbContext, and migrations	9
Step 3: Configure CORS in the API	9
Step 4: Register HttpClient in Blazor	9
Step 5: Create a test endpoint and rewire Admin.razor	10
Step 6: Verify both projects run simultaneously	10
8 Architecture Reference	11
8.1 Project Separation	11
8.2 Why Blazor Server Stays as Blazor Server	11
8.3 Connecting the Two Projects	11
8.4 SQLite in Both Environments	12

9	Revised 4-Week Build Plan	13
9.1	Week 1 — Foundations (Resume from Day 5)	13
9.2	Week 2 — Core Game Loop	14
9.3	Week 3 — Spells & Real-Time Multiplayer	15
9.4	Week 4 — Polish & Deployment	17
A	Key Commands Reference	20
B	Database Schema	21
C	Learning Resources	21

Abstract

Clash of Codes is a multiplayer competitive programming game that adapts the high-stakes, real-time resource management of the Clash Royale format into an algorithmic battleground. Players match globally or challenge friends in private rooms, balancing fast-paced code composition, multiple-choice questions, and disruptive environmental abilities called **Spells**.

This document covers the complete game design, design improvements, the two-project architecture reference, a migration guide for the existing codebase, and the full revised day-by-day build plan.

1 The Core Game Loop & Resource Economy

1.1 Battle Elixir (BE)

- **Capacity:** Players start at 0 BE; hard cap is **10 BE**.
- **Passive generation:** 1 BE every 10 seconds. In the final 60 seconds (**Double Elixir mode**): 2 BE every 10 seconds.
- **Active generation:**

Milestone	BE Bonus
Successful compilation	+1
Flawless first-submission run	+3
Correct MCQ answer	+1

1.2 Rank Points (RP) and the Conversion Mechanic

Final RP Awarded = Match Base Victory RP + (Unspent BE at match end × δ) $\delta = 5$

After 3 consecutive losses, the RP penalty for the next match is halved.

2 Match Structure & Victory Conditions

2.1 The Three-Track Node Setup

1. **Left Track (Easy):** Yields 1 Crown on completion.
2. **Right Track (Medium):** Yields 1 Crown on completion.
3. **Centre Track (Hard):** King’s Tower — completing this triggers an instant **3-Crown absolute victory**, overriding all other conditions.

2.2 Node Workflow

Stage I: Code Composition (pass public test cases) → **Stage II:** MCQ Stream (3–5 questions)

Wrong code submissions deduct flat points; wrong MCQ answers apply negative scaling penalties.

3 The Spellbook Mechanics

Players assemble a loadout of exactly **4 Spells** (2 Offensive + 2 Defensive).

Launch with **6 spells only** (3 offensive + 3 defensive). Recommended first set: Fog of War, Time Warp, MCQ Barrage + Lint Shield, Garbage Collector, Ctrl+Z Rollback.

3.1 Offensive Spells

Spell	BE	Effect
Fog of War	2	Hides compiler line numbers for 45s; opponent sees only “Compilation Failed”.
Time Warp	3	Opponent’s timer spins at double speed visually for 30s.
Chaff	4	Obfuscates 50% of sample I/O values on opponent’s screen.
EMP (Flicker)	4	Scrambles editor line numbers and flashes the screen for 15s.
Syntax Poison	4	Switches opponent’s IDE to ultra-low contrast theme for 30s.
MCQ Barrage	5	Injects 2 high-difficulty MCQ items into opponent’s queue.
The Typo Curse	5	30% chance each semicolon maps to a Greek question mark for 30s.
Bracket Bandit	6	Deletes all trailing },],) from opponent’s editor buffer.
Memory Leak	6	Caps execution runtime for 45s, causing artificial TLE.
Code Swap	8	Replaces opponent’s current problem with a new equal-tier one.

3.2 Defensive Spells

Spell	BE	Effect
Clock Sync	1	Re-aligns local timer with true server timestamp.
Lint Shield	2	Immune to flickering, theme changes, typo curses for 60s.
Debugger Sight	3	Permanently reveals 2 hidden edge-case inputs.
Sanitize Input	3	Purges active Chaff obfuscations immediately.

Spell	BE	Effect
Ctrl+Z (Roll-back)	4	3s reactive window restoring code buffer after Code Swap.
Firewall	4	Selected problem node cannot be altered for 90s.
Garbage Collector	5	Purges all active debuffs and visual distortions.

4 Matchmaking & Arenas

4.1 Ranked Ladder

Pairs players within ± 200 RP. Questions selected randomly across sub-disciplines matched to the arena tier.

4.2 Custom Lobbies

- **Room Codes:** 6-character alphanumeric key for direct peer connection.
- **Configuration:** Topic filter, difficulty cap, Elixir mode (Normal / Infinite 7x).

4.3 Syntax Stages (Arenas)

Arena	Name & RP Range	Themes
1	Spaghetti Junction (0–499)	Primitive ops, arrays, string parsing
2	Parse Pit (500–999)	Sorting, two-pointer, prefix sums
3	Stack Shack (1000–1499)	Stacks, queues, sliding window
4	Heap Hollow (1500–1999)	Binary search, heaps, greedy
5	The Stack Overflow (2000–2499)	Recursive trees, heavy graphs
6	Pointer Purgatory (2500–2999)	Custom objects, DFS/BFS mastery
7	Bitwise Badlands (3000–3499)	Bit manipulation, advanced graphs
8	Complexity Citadel (3500–3999)	Dynamic programming (medium-hard)
9	Recursion Realm (4000–4999)	Advanced DP, segment trees
10	Kernel Palace (5000+)	Systems architecture, scheduling

Each new arena unlocks one additional spell in the Spellbook.

5 Recommended Design Improvements

5.1 Spell Cooldown

Add a **30–60 second per-slot cooldown** after casting to prevent burst spell combos (e.g., Bracket Bandit + Code Swap in rapid succession).

5.2 Active Debuff Cap

At most **2 active offensive debuffs** on the opponent at any time. A third spell replaces the oldest.

5.3 Disconnect Handling

A player disconnected for 60+ seconds auto-forfeits; opponent wins with a partial RP bonus.

5.4 Spell Notifications

Every incoming spell displays a toast, e.g.:

“Opponent cast Fog of War — compiler messages hidden for 45s”

6 Current Project State (End of Day 4)

Every file already built is kept. Models, DbContext, migrations, and all Razor pages move to their correct project — they are not deleted or rewritten.

6.1 What You Have and Where It Goes

File / Folder	Destination
Models/ (all 6 files)	Move to API project
Data/AppDbContext.cs	Move to API project
Data/DataExtension.cs	Move to API project
Migrations/	Move to API project
ClashOfCodes.db	Move to API project folder
Pages/Admin.razor	Keep in Blazor (rewire to API call)
Pages/Home, Profile, Game.razor	Keep in Blazor
App.razor, Routes.razor	Keep in Blazor
Program.cs (Blazor)	Strip DB/Identity, keep routing only

7 Migration Guide (Day 4.5)

This takes approximately 1–2 hours and is done only once. After this the architecture is correct for everything that follows.

Step 1: Create the API project

Open a terminal in the solution root (the folder containing `ClashOfCodes`). Run:

```
dotnet new webapi -o ClashOfCodes.API
dotnet sln add ClashOfCodes.API/ClashOfCodes.API.csproj
```

Your folder structure should now be:

```
CodingGame/
  ClashOfCodes/          <- existing Blazor project
  ClashOfCodes.API/      <- new API project
  ClashOfCodesSolution.slnx
```

Step 2: Move models, DbContext, and migrations

Copy (do not delete yet) these items from `ClashOfCodes/` into `ClashOfCodes.API/`:

```
Models/          -> ClashOfCodes.API/Models/  
Data/            -> ClashOfCodes.API/Data/  
Migrations/      -> ClashOfCodes.API/Migrations/  
ClashOfCodes.db -> ClashOfCodes.API/ClashOfCodes.db
```

Add EF Core + SQLite to the API project:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite  
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Update all namespaces in the copied files from `ClashOfCodes` to `ClashOfCodes.API`. Register the `DbContext` in the API's `Program.cs`:

```
builder.Services.AddDbContext<AppDbContext>(o =>  
    o.UseSqlite("Data Source=ClashOfCodes.db"));
```

Step 3: Configure CORS in the API

In the API's `Program.cs`, add CORS before `app.Run()`:

```
builder.Services.AddCors(o => o.AddPolicy("Blazor", p =>  
    p.WithOrigins("https://localhost:7200") // your Blazor  
      port  
      .AllowAnyHeader()  
      .AllowAnyMethod()  
      .AllowCredentials()))); // required for SignalR later  
  
// ...  
  
app.UseCors("Blazor");
```

Check your actual Blazor port in `ClashOfCodes/Properties/launchSettings.json` and match it here.

Step 4: Register HttpClient in Blazor

In the Blazor project's `Program.cs`:

```
builder.Services.AddScoped(sp =>  
    new HttpClient  
    {  
        BaseAddress = new Uri("https://localhost:7100")  
        // match your API port from its launchSettings.json  
    });
```

Step 5: Create a test endpoint and rewire Admin.razor

In the API, create `Controllers/ProblemsController.cs`:

```
[ApiController, Route("api/[controller]")]
public class ProblemsController : ControllerBase
{
    private readonly AppDbContext _db;
    public ProblemsController(AppDbContext db) { _db = db; }

    [HttpGet]
    public async Task<IActionResult> Get()
        => Ok(await _db.Problems.ToListAsync());
}
```

In Blazor's `Admin.razor`, replace any direct `AppDbContext` injection:

```
@inject HttpClient Http

@code {
    List<Problem>? problems;

    protected override async Task OnInitializedAsync()
        => problems = await Http
            .GetFromJsonAsync<List<Problem>>("api/problems");
}
```

When the Admin page loads data from the API successfully, the migration is complete. Now delete the original `Models/`, `Data/`, and `Migrations/` folders from the Blazor project, and remove any `AddDbContext` calls from the Blazor `Program.cs`.

Step 6: Verify both projects run simultaneously

Open two terminals and run both:

```
# Terminal 1
cd ClashOfCodes.API && dotnet run

# Terminal 2
cd ClashOfCodes && dotnet run
```

Open the Blazor app in the browser, navigate to the Admin page, and confirm that it loads data fetched from the API. If it does, you are ready to resume Day 5.

8 Architecture Reference

8.1 Project Separation

ClashOfCodes.API	ClashOfCodes (Blazor)
ASP.NET Core 8 Web API	Blazor Server (existing project)
EF Core + SQLite	HttpClient (all API calls)
ASP.NET Identity	Custom AuthstateProvider
JWT token generation	JWT stored in localStorage
SignalR Hub	HubConnectionBuilder
Judge0 service	Monaco Editor (JS interop)
Matchmaking BackgroundService	Spell hotbar + game UI
Azure App Service	Azure App Service

Communication: REST over HTTP/JSON for data + WebSockets via SignalR for real-time events

8.2 Why Blazor Server Stays as Blazor Server

The existing project is Blazor Server. There is no need to convert it to WebAssembly — Blazor Server can call a separate Web API perfectly well using `HttpClient` and `HubConnectionBuilder`. For deployment, Blazor Server goes to Azure App Service (not Azure Static Web Apps, which is only for WASM since it outputs purely static files).

8.3 Connecting the Two Projects

REST calls via `HttpClient`

```
// In any Blazor component:
@Inject HttpClient Http

var problems = await Http
    .GetFromJsonAsync<List<ProblemDto>>("api/problems");

await Http.PostAsJsonAsync("api/submit", new SubmitRequest
{
    Code = code, Language = "csharp", ProblemId = id
});
```

JWT attached automatically via `AuthorizationMessageHandler`

```
public class JwtAuthHandler : DelegatingHandler
{
    private readonly IJSRuntime _js;
    public JwtAuthHandler(IJSRuntime js) { _js = js; }
```

```
protected override async Task<HttpResponseMessage>
    SendAsync(HttpRequestMessage req,
        CancellationToken ct)
{
    var token = await _js.InvokeAsync<string>(
        "localStorage.getItem", "jwt");
    if (!string.IsNullOrEmpty(token))
        req.Headers.Authorization =
            new AuthenticationHeaderValue("Bearer", token);
    return await base.SendAsync(req, ct);
}
```

SignalR connection with JWT

```
var connection = new HubConnectionBuilder()
    .WithUrl(apiBaseUrl + "/hubs/game", opts =>
    {
        opts.AccessTokenProvider = async () =>
            await _js.InvokeAsync<string>(
                "localStorage.getItem", "jwt");
    })
    .WithAutomaticReconnect()
    .Build();

await connection.StartAsync();

// Listen for events from the server
connection.On<SpellDto>("SpellReceived", ApplySpell);

// Send events to the server
await connection.SendAsync("CastSpell", spellId);
```

8.4 SQLite in Both Environments

SQLite is a single file (**ClashOfCodes.db**). Unlike SQL Server, it requires no server process. The connection string is simply a file path:

```
// In API Program.cs
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlite("Data Source=ClashOfCodes.db"));
```

For Azure deployment, copy the **.db** file to the App Service storage or switch to a managed database (Azure SQL or PostgreSQL) before going to production. SQLite is perfectly fine for development and testing throughout the 4-week build.

9 Revised 4-Week Build Plan

API Work in the ASP.NET Web API project **Blazor** Work in the Blazor Server project **Both** Work spans both projects

Week	Theme	End-of-week milestone
1	Foundations (resume Day 5)	Auth working end-to-end with JWT across both projects
2	Core game loop	Full solo play: code submit, MCQ, BE economy, result screen
3	Multiplayer	SignalR real-time match, 6 spells, match-making queue
4	Polish & deploy	Leaderboard, private lobbies, UI polish, both live on Azure

9.1 Week 1 — Foundations (Resume from Day 5)

Models, DbContext, migrations, SQLite database, and Admin page are complete. Complete the 5-step migration guide in Section 7 before starting Day 5.

Days 5–7: JWT authentication end-to-end

Day 5 **API: Identity + JWT token generation** **API**

Add `Microsoft.AspNetCore.Identity.EntityFrameworkCore` and `Microsoft.AspNetCore.Authentication.JwtBearer` to the API project. Extend your existing `User` model to inherit from `IdentityUser` (add your custom fields: `RankPoints`, `CurrentArena`, `Wins`, `Losses`, `SelectedSpellsJson`). Create `AuthController` with `POST /api/auth/register` and `POST /api/auth/login`. On login, generate and return a JWT using `JwtSecurityTokenHandler`. Run a new migration: `dotnet ef migrations add AddIdentity`. Test both endpoints in the Swagger UI at `/swagger`.

Day 6 **Blazor: login, register pages, store JWT** **Blazor**

Build `Login.razor` and `Register.razor` using Blazor's `EditForm` with `DataAnnotationsValidator`. On successful login, store the JWT in `localStorage` via `IJSRuntime: await JS.InvokeVoidAsync("localStorage.setItem", "jwt", token)`. Write a custom `AuthStateProvider` that reads the token, parses the user's claims, and exposes the identity to Blazor. Wire up `<AuthorizeView>` in the navbar to show login/logout based on auth state.

Day 7 Attach JWT to all requests + Profile page **Both**

Write a `JwtAuthHandler` (see architecture section) that automatically reads the JWT from `localStorage` and attaches it as a `Bearer` header on every `HttpClient` request. Register it in Blazor's `Program.cs`. Add `[Authorize]` to protected API endpoints. Rewire your existing `Profile.razor` to call `GET /api/users/me` — a protected endpoint returning RP, arena, wins, and losses — instead of querying the database directly.

9.2 Week 2 — Core Game Loop

Full solo loop: load problem, write code, submit to API, answer MCQs, earn BE, see a result screen with RP update — all via HTTP calls.

Days 8–9: Code editor and Judge0 submission

Day 8 API: `JudgeService` + submit endpoint **API**

Create `Services/JudgeService.cs` in the API that calls the Judge0 CE API (free tier, 100 submissions/day at <https://judge0.com>) via `HttpClient`. Register it in `Program.cs` with `builder.Services.AddScoped<JudgeService>()`. Create `POST /api/submit` receiving code + language + `problemId`, fetching the problem's test cases from the database, running them through Judge0, and returning pass/fail per test case plus a BE bonus value. Blazor never calls Judge0 directly.

Day 9 Blazor: Monaco Editor + submit flow **Both**

Embed Monaco Editor in Blazor via JS interop loaded from CDN. Learn `IJSRuntime` — you use it to initialise the editor, get the current code, and later to apply spell effects. Build `Components/CodeEditor.razor` with a `GetCode()` method. Wire a Submit button that calls `POST /api/submit` and renders the returned test case results (pass/fail per case, total passed count).

Days 10–11: MCQ system and BE economy

Day 10 API: MCQ endpoints with server-side validation **API**

Add `GET /api/mcq/{problemId}` returning 3–5 questions *without* the correct answer index — never send answers to the client. Add `POST /api/mcq/answer` receiving the player's chosen index and returning correct/wrong plus a BE delta value. Your existing `McqQuestion` model is already in the database from Day 3.

Day 11 Blazor: MCQ panel, BE bar, crown tracker

Blazor

Build `Components/McqPanel.razor`: fetches questions from the API, displays them one at a time, POSTs the answer, and applies the returned BE delta to local component state using `StateHasChanged()`. Build a BE bar component (0–10 fill, gradient on Double Elixir). Build a crown tracker (3 slots, filled on node completion). Add passive BE generation using a `System.Threading.Timer` in the game page's code-behind (+1 every 10 seconds, capped at 10).

Days 12–14: Full solo game screen and result**Day 12** Wire up `Game.razor` with all components

Blazor

Your `Game.razor` page already exists — now fill it in. Use CSS Grid for the layout: left column for the problem statement and code editor, right column for the MCQ panel, crown tracker, BE bar, and a 5-minute countdown timer. On page load, fetch 3 problems (1 easy, 1 medium, 1 hard) from `GET /api/problems/match`. Wire all components together and play the full solo loop end-to-end before adding multiplayer.

Day 13 API + Blazor: result endpoint and RP formula

Both

Create `POST /api/matches/result` in the API. It receives crowns won and unspent BE, computes $\text{FinalRP} = \text{BaseRP} + (\text{UnspentBE} \times 5)$, updates `RankPoints` and `CurrentArena` in the database, and returns the updated values plus a `promoted` flag if an arena threshold was crossed. Build the result screen in Blazor: crowns, RP gained, new total, and an arena promotion animation when `promoted` is true.

Day 14 Buffer and preparation for SignalR

Both

Fix any bugs from the solo loop. Test the complete flow: register, login, play, result, profile RP updated. Read the SignalR documentation before Week 3: <https://learn.microsoft.com/aspnet/core/tutorials/signalr-blazor>. Note: you are using `HubConnectionBuilder` in both Blazor Server and Blazor WASM — the syntax is the same but authentication attachment differs slightly from `@inject`-based approaches.

9.3 Week 3 — Spells & Real-Time Multiplayer

Two players in the same match, seeing each other's progress live, casting spells, and found via the ranked queue. Days 15–16 are the hardest days of the project.

Days 15–17: SignalR real-time match

Day 15 API: GameHub and in-memory match state API

Add `Microsoft.AspNetCore.SignalR` to the API project. Create `Hubs/GameHub.cs`. Store live match state in a server-side `ConcurrentDictionary<string, MatchState>` (a simple in-memory dictionary — no database writes during a match, only on completion). Map `/hubs/game` in `Program.cs`. Update the CORS policy to include `.AllowCredentials()` — required for SignalR WebSocket connections.

Day 16 Blazor: connect to hub and opponent progress bar Blazor

Add `Microsoft.AspNetCore.SignalR.Client` to the Blazor project. Build the `HubConnectionBuilder` with the JWT token attached via `AccessTokenProvider`. On each successful test-case submission, call `connection.SendAsync("UpdateProgress", matchId, percentage)`. Listen for `connection.On("OpponentProgress", pct => ...)` and update the opponent's progress bar. Test with two browser windows simultaneously. This is the hardest day of the project — take your time.

Day 17 Double Elixir, match end, and disconnect handling Both

At T-60s the server broadcasts `DoubleElixirStarted`; both clients double their passive BE rate. Server detects win conditions: 3-crown instant win, or most crowns when the timer hits zero (tie-breaker: most test cases passed). Broadcasts `MatchEnded` with winner info. Both clients disconnect from the hub and call `POST /api/matches/result`. If a player disconnects for 60+ seconds, they auto-forfeit and the opponent wins with a partial RP bonus.

Days 18–21: Spellbook and matchmaking queue

Day 18 API: server-authoritative CastSpell hub method API

Add `CastSpell(string matchId, int spellId)` to `GameHub`. Server-side validation: check the caster has enough BE and no active cooldown (track cooldown timestamps in the in-memory `MatchState`). Deduct BE. Broadcast the spell effect only to the target player's connection ID using `Clients.Client(targetConnectionId).SendAsync("SpellReceived", spellId)`. Spell logic is always server-authoritative — the client cannot cheat.

Day 19 Blazor: receive and apply 3 offensive spell effects Blazor

Listen for `connection.On("SpellReceived", spellId => ApplySpell(spellId))`. Implement the first 3 offensive effects: **Fog of War** (JS interop hides the compiler output section), **Syntax Poison** (inject a CSS class that overrides Monaco's theme with a near-invisible low-contrast palette), **MCQ Barrage** (push 2 extra items onto the local MCQ queue). Build a toast notification component: each incoming spell shows name and duration.

Day 20 Blazor: 3 defensive spells and spell hotbar UI Both

Implement the first 3 defensive spells: **Lint Shield** (set a local `bool immuneToVisual` flag, checked before applying visual effects), **Garbage Collector** (clear all active debuff flags and remove any injected CSS classes), **Rollback** (save a Monaco code snapshot every 30s; on Cast, restore the latest snapshot within a 3-second window following a Code Swap). Build the spell hotbar: 4 cards showing name, BE cost, a cooldown ring animation using a CSS `conic-gradient`, and a Cast button that calls `CastSpell` on the hub.

Day 21 API + Blazor: matchmaking queue Both

Build `Services/MatchmakingQueue.cs` as a `BackgroundService` in the API. `POST /api/matchmaking/join` adds the authenticated user to the queue. Every 5 seconds the service pairs players within ± 200 RP, creates a `Match` record in the database, assigns 3 random problems (1 easy, 1 medium, 1 hard filtered by arena), and broadcasts `MatchFound` with the `matchId` to both players via `SignalR`. Blazor listens for `MatchFound` and redirects to `/game/{matchId}`.

9.4 Week 4 — Polish & Deployment

Global leaderboard, arena system, private lobbies, UI polish, and both projects live on Azure with correct CORS and environment variables configured.

Days 22–25: Leaderboard, arenas, and private lobbies

Day 22 API + Blazor: arena promotion system Both

Define the 10 RP thresholds in a static config class in the API. In the match result endpoint, after updating `RankPoints`, check if the new value crosses an arena boundary, update `CurrentArena`, and include a `promoted: true` flag in the response. Blazor plays a promotion animation (CSS transition) when it receives this flag. Matchmaking uses `CurrentArena` to filter problem difficulty.

Day 23 API + Blazor: global leaderboard Both

Add `GET /api/leaderboard?page=1` returning the top 50 users ordered by `RankPoints` descending, with arena badge, win rate, and RP. Use `IQueryable` with `Skip/Take` for pagination. Add `GET /api/leaderboard/me` returning the current user's global rank even if outside the top 50 (use `COUNT` of users with higher RP). Fill in the existing `Leaderboard.razor` placeholder page in Blazor.

Day 24 **API: private room endpoints** **API**

Using your existing Room model (already in the database), add: POST /api/rooms (creates a room, returns a 6-character random code), POST /api/rooms/join (finds room by code, adds player), POST /api/rooms/start (host triggers match creation and broadcasts MatchFound to both players). Room config stored as JSON: topic filter, difficulty cap, Elixir mode. All endpoints require [Authorize].

Day 25 **Blazor: lobby waiting room UI** **Both**

Build a waiting room page: both players shown with avatar initials (first letter of username in a coloured circle), a Ready toggle button, a live chat panel using SignalR (reuse GameHub with a lobby group prefix), and the room config summary. Host sees a Start button once both players are ready. A share button copies the room link to the clipboard via navigator.clipboard.writeText through JS interop.

Days 26–27: UI polish**Day 26** **Blazor: global visual design pass** **Blazor**

Define a dark colour theme in wwwroot/css/app.css using CSS custom properties (-color-primary, -color-surface, etc.). Add entrance animations for spell cards (CSS @keyframes slideIn) and crown capture events. Design arena badge icons as inline SVG. Add a pulse animation to the BE bar during Double Elixir mode. All styling remains in the Blazor project — the API has no UI.

Day 27 **Blazor: game screen layout and keyboard shortcuts** **Blazor**

Refine the game page CSS Grid: left panel (problem statement + code editor), right panel (opponent progress + spell hotbar). Add a collapsible problem statement toggle. Add a Ctrl+Enter keyboard shortcut to submit code via JS interop: await JS.InvokeVoidAsync("addSubmitShortcut", dotNetRef). Add loading spinner components to all HttpClient calls so the UI never appears frozen during API latency.

Days 28–30: Testing and deployment**Day 28** **End-to-end testing with two accounts** **Both**

Open two browsers (Chrome and Firefox). Register two separate accounts. Play a full ranked match from queue through to result: verify crown capture, spell effects on both sides, BE deduction, RP update, and arena promotion. Log every bug as a GitHub issue. Verify JWT expiry is handled gracefully (redirect to login, not a crash or blank screen).

Day 29 Bug fixes and error handling Both

Fix all bugs logged on Day 28. In Blazor, add `<ErrorBoundary>` wrappers to the game page components. Wrap all `HttpClient` calls in `try/catch` with user-friendly error messages. Handle 401 responses by clearing the stored JWT and redirecting to the login page. Handle Judge0 API timeouts in the API by returning a structured error response rather than a 500 crash.

Day 30 Deploy both projects to Azure Both

Deploy **ClashOfCodes.API** to Azure App Service (Free or Basic tier). Deploy **ClashOfCodes** (Blazor Server) to a second Azure App Service — Blazor Server requires a server process; it cannot use Azure Static Web Apps. Update the API's `CORS WithOrigins` to the live Blazor App Service URL. Update the Blazor `HttpClient` base address to the live API URL (use environment-specific `appsettings.json`). Store all secrets as Azure environment variables: SQLite path, JWT secret key, Judge0 API key. Never commit secrets to source code.

A Key Commands Reference

```
# Create the API project and add to solution
dotnet new webapi -o ClashOfCodes.API
dotnet sln add ClashOfCodes.API/ClashOfCodes.API.csproj

# EF Core packages for the API
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package
    Microsoft.AspNetCore.Identity.EntityFrameworkCore

# JWT authentication (API)
dotnet add package
    Microsoft.AspNetCore.Authentication.JwtBearer

# SignalR (API)
dotnet add package Microsoft.AspNetCore.SignalR

# SignalR client (Blazor)
dotnet add package Microsoft.AspNetCore.SignalR.Client

# EF Core migrations (run from ClashOfCodes.API folder)
dotnet ef migrations add AddIdentity
dotnet ef database update

# Run both projects simultaneously (two terminals)
cd ClashOfCodes.API && dotnet run      # Terminal 1
cd ClashOfCodes && dotnet run          # Terminal 2

# Publish API for Azure
dotnet publish ClashOfCodes.API -c Release -o ./api-publish

# Publish Blazor for Azure
dotnet publish ClashOfCodes -c Release -o ./blazor-publish
```

B Database Schema

Table	Key Fields
AspNetUsers	(Identity default) + RankPoints, CurrentArena, Wins, Losses, SelectedSpellsJson
Problems	Id, Title, Description, Difficulty (1/2/3), Topic, TestCasesJson, HiddenTestCasesJson
McqQuestions	Id, ProblemId (FK), QuestionText, OptionsJson, CorrectIndex
Matches	Id, CreatedAt, EndedAt, WinnerId (FK), IsRanked, RoomCode
MatchPlayers	MatchId (FK), UserId (FK), CrownsEarned, FinalBE, RpChange, SpellsUsedJson
Rooms	Id, Code, HostUserId (FK), ConfigJson, Status (Waiting/Active/-Closed)

C Learning Resources

- Blazor Server docs: <https://learn.microsoft.com/aspnet/core/blazor/>
- SignalR with Blazor tutorial: <https://learn.microsoft.com/aspnet/core/tutorials/signalr-blazor>
- EF Core with SQLite: <https://learn.microsoft.com/ef/core/providers/sqlite/>
- JWT in ASP.NET Core: <https://learn.microsoft.com/aspnet/core/security/authentication/jwt-authn>
- Judge0 CE API: <https://judge0.com>
- Monaco Editor: <https://microsoft.github.io/monaco-editor/>
- Azure App Service: <https://azure.microsoft.com/products/app-service/>